



**UNIVERSIDAD DE
COSTA RICA**
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 1: Fundamentos de Git, Maven y Comunicación Cliente-Servidor en Java Nativo

Resumen

Esta práctica guiada tiene como objetivo introducir al estudiantado en los flujos de trabajo profesionales de control de versiones utilizando Git y GitHub, la automatización en la construcción de proyectos y gestión de dependencias con Maven, y la implementación básica de una aplicación web sin frameworks utilizando las API nativas de Java SE. Se busca establecer una base técnica sólida sobre la cual, en las próximas prácticas, se incorporarán frameworks más avanzados como Spring Boot, Spring Beans, JPA, Hibernate y Angular.

Concepto Principal (El 'Qué')

En el desarrollo de software web empresarial de pila completa (*full-stack*), existen tres pilares metodológicos y tecnológicos que estructuran cualquier proyecto moderno:

1. Control de Versiones (Git & GitHub)

Git es un sistema descentralizado de control de versiones que registra los cambios realizados en el código fuente de un proyecto a lo largo del tiempo. Permite a los desarrolladores aislar sus contribuciones mediante **ramas (branches)**, facilitando el desarrollo paralelo de nuevas características sin alterar la línea principal de producción. **GitHub** actúa como el repositorio remoto y plataforma de colaboración, permitiendo la integración de código a través de **solicitudes de extracción (Pull Requests)** y revisiones de código.

2. Gestión de Ciclo de Vida y Dependencias (Maven)

Apache Maven es una herramienta de automatización y gestión de proyectos basada en el concepto de un **Modelo de Objetos de Proyecto (POM)**, materializado en el archivo `pom.xml`. Maven automatiza el ciclo de vida del build (compilación, pruebas unitarias, empaquetado, instalación y despliegue) y resuelve de manera transitiva las dependencias externas desde repositorios centralizados, eliminando la necesidad de importar archivos binarios (`.jar`) manualmente.

3. Comunicación asíncrona mediante JSON sobre HTTP

La arquitectura de la Web moderna separa de forma estricta la interfaz de usuario (**Frontend**) de la lógica del servidor (**Backend**). El protocolo **HTTP (Hypertext Transfer Protocol)** opera sobre un ciclo de solicitud/respuesta (*Request/Response*). Para lograr dinamismo sin recargar la página completa, el frontend utiliza la **Fetch API** para enviar solicitudes asíncronas POST al backend. Los datos se serializan en formato **JSON (JavaScript Object Notation)**, un estándar ligero de intercambio de datos estructurados de clave-valor.

El backend procesa la solicitud, y para que el navegador interprete correctamente el resultado, se deben especificar los **tipos MIME** adecuados en los encabezados HTTP (ej. `text/html` para documentos, `text/css` para estilos y `application/json` para respuestas de datos estructurados).

Analogía (La Intuición)

- **Git / GitHub:** Imagine que escribe un contrato legal complejo en conjunto con otros abogados. En lugar de enviarse archivos renombrados como `contrato_final_v2_editado.docx`, utilizan un libro de registro de firmas e historial de cambios donde cada cambio debe ser aprobado por el editor jefe (GitHub) antes de integrarse de forma definitiva al documento principal.
- **Maven:** Imagine que es el chef ejecutivo de un restaurante de alta cocina. Para preparar un platillo complejo, usted no viaja por el país buscando cada ingrediente; en su lugar, escribe una receta formal donde lista los proveedores y los ingredientes necesarios (`pom.xml`). Un asistente automatizado (Maven) va al almacén, consigue exactamente lo que pidió y lo prepara en su cocina siguiendo las fases ordenadas de la receta.
- **HTTP y JSON:** Imagine que envía una carta certificada a una oficina postal. La oficina postal exige que la carta tenga una etiqueta que diga qué tipo de contenido es (Encabezado `Content-Type`) para saber cómo clasificarla. El contenido adentro es un formulario estandarizado escrito en un idioma común que tanto el remitente como el destinatario pueden leer sin importar su nacionalidad (JSON).

Ejemplo de Código e Implementación (El 'Cómo')

A continuación, usted creará paso a paso un proyecto desde cero, configurando el repositorio en Git, inicializando la estructura de Maven de forma manual y codificando un servidor HTTP nativo en Java que interactúa con un formulario web.

Paso 1: Inicialización y Flujo de Git desde Cero

Antes de escribir código, configure su repositorio local. Abra una terminal en su directorio de trabajo y ejecute los siguientes pasos:

1. Configure sus credenciales globales (reemplace los valores por sus datos reales de GitHub):

```
git config --global user.name "Su Nombre"
git config --global user.email "su-correo@mail.com"
```

1. Inicialice el repositorio en la carpeta raíz del proyecto:

```
git init
```

1. Configure el archivo `.gitignore` para omitir los archivos generados por el compilador y el entorno. Cree un archivo llamado `.gitignore` en la raíz y agregue el siguiente contenido:

```
/target/  
/.settings/  
/.classpath  
/.project  
/.idea/  
/*.iml  
.DS_Store
```

Uso del Panel de Control de Fuentes en VS Code

Visual Studio Code provee una interfaz gráfica intuitiva para realizar operaciones de Git. Para interactuar con ella:

1. Abra la pestaña **Source Control** utilizando el atajo de teclado `Ctrl + Shift + G`.
2. Verá una sección de **Changes** con todos los archivos modificados. Puede agregar un archivo al área de preparación (*Staging Area*) haciendo clic en el icono `+` (Stage Changes) al lado del nombre del archivo.
3. Escriba un mensaje descriptivo en la caja de texto superior y presione el botón **Commit** (o presione el checkmark) para guardar los cambios localmente.

Consejos de Integración con el Copiloto de IA (Google Antigravity)

Usted puede apoyarse en su asistente de IA integrado en el IDE para agilizar tareas repetitivas y entender errores de Git utilizando prompts estructurados:

- **Para redactar commits semánticos:**
"Asistente, revise mis cambios actuales en el Staging Area de Git y sugiera un mensaje de commit semántico bajo la convención Conventional Commits."
- **Para resolver dudas de Git:**
"Asistente, ejecuté el comando git push y obtuve el siguiente error: [pegar error]. Explique

cuál es la causa raíz y proporcione los pasos exactos en terminal para solucionarlo."

Paso 2: Creación de la Estructura de Maven desde Cero

Para comprender cómo funciona Maven sin la abstracción automática de los asistentes de los IDEs, usted construirá el archivo de configuración POM y los directorios de manera manual.

Estándar de Nomenclatura del Curso:

Para todos los proyectos y entregables del curso, se establece que tanto el identificador de grupo en Maven (`groupId`) como los paquetes de Java deben seguir obligatoriamente la estructura estándar:

`cr.ac.ucr.paraíso.ie.<id>.<package_name>`

Donde `<id>` corresponde a su carné universitario (ej. b98765) y `<package_name>` al nombre del entregable o módulo (para esta práctica, `practica1`). En los ejemplos siguientes, reemplace la palabra `carnet` con su propio carné en minúsculas.

1. Cree el archivo `pom.xml` en la ruta `backend/pom.xml` con el siguiente contenido estructurado. Note que incluimos la biblioteca **Gson** de Google para facilitar la conversión de texto JSON a objetos Java:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>cr.ac.ucr.paraíso.ie.carnet.practica1</groupId>
  <artifactId>practica-1</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <!-- Dependencia para serialización y deserialización JSON -->
    <dependency>
```

```
<groupId>com.google.code.gson</groupId>
<artifactId>gson</artifactId>
<version>2.10.1</version>
</dependency>
</dependencies>
</project>
```

1. Cree la estructura jerárquica de carpetas de Maven y el directorio público para los archivos HTML/CSS. Ejecute los siguientes comandos en la consola (o créelos desde el explorador de archivos de su IDE):

```
# Crear carpeta para el Backend y su estructura Java (reemplace con su carné)
mkdir -p backend/src/main/java/cr/ac/ucr/paraíso/ie/carnet/practica1

# Crear carpeta para el Frontend estático
mkdir -p frontend
```

Paso 3: Implementación del Servidor HTTP Java Nativo (Backend)

Cree el archivo de código Java `AppServer.java` en la ruta `backend/src/main/java/cr/ac/ucr/paraíso/ie/carnet/practica1/AppServer.java`.

Este servidor realiza dos tareas fundamentales: sirve archivos estáticos de la carpeta externa `frontend/` y expone un endpoint API REST `/api/submit` que recibe datos JSON vía POST, los deserializa y retorna un saludo personalizado.

```
package cr.ac.ucr.paraíso.ie.carnet.practica1;

import com.google.gson.Gson;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;
import java.io.*;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;

public class AppServer {
```

```

    public static void main(String[] args) throws IOException {
        // Inicializar servidor en el puerto 8080
        HttpServer server = HttpServer.create(
            new InetSocketAddress(8080), 0
        );

        // Mapear rutas
        server.createContext("/", new StaticFileHandler());
        server.createContext("/api/submit", new ApiHandler());

        server.setExecutor(null); // Usar ejecutor por defecto
        System.out.println("Servidor en: http://localhost:8080");
        server.start();
    }

    // Handler para servir HTML y CSS
    static class StaticFileHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            String path = exchange.getRequestURI().getPath();
            if (path.equals("/")) {
                path = "/index.html";
            }

            // Buscar los archivos en la carpeta frontend vecina del backend
            File file = new File("../frontend" + path);
            if (!file.exists() || file.isDirectory()) {
                String err = "404 - Archivo no encontrado";
                exchange.sendResponseHeaders(404, err.length());
                OutputStream os = exchange.getResponseBody();
                os.write(err.getBytes());
                os.close();
                return;
            }

            // Detectar el tipo MIME correcto
            String mime = "text/plain";
            if (path.endsWith(".html")) mime = "text/html";
            else if (path.endsWith(".css")) mime = "text/css";
            else if (path.endsWith(".js")) mime = "text/javascript";

            exchange.getResponseHeaders().set("Content-Type", mime);
            byte[] bytes = Files.readAllBytes(file.toPath());
            exchange.sendResponseHeaders(200, bytes.length);

            OutputStream os = exchange.getResponseBody();

```

```

        os.write(bytes);
        os.close();
    }
}

// Handler para procesar el JSON del Frontend
static class ApiHandler implements HttpHandler {
    private final Gson gson = new Gson();

    @Override
    public void handle(HttpExchange exchange) throws IOException {
        // Solo permitir solicitudes de tipo POST
        if (!"POST".equalsIgnoreCase(exchange.getRequestMethod())) {
            exchange.sendResponseHeaders(405, -1); // 405 Method Not Allowed
            return;
        }

        // Leer cuerpo de la petición HTTP
        InputStreamReader reader = new InputStreamReader(
            exchange.getRequestBody(), StandardCharsets.UTF_8
        );

        // Convertir JSON a objeto Java
        UserData data = gson.fromJson(reader, UserData.class);
        reader.close();

        // Lógica de negocio (Crear saludo)
        String greeting = "¡Hola, " + data.nombre + "! "
            + "Hemos registrado su correo: " + data.correo;

        JsonResponse responseObj = new JsonResponse(
            "success", greeting
        );
        String responseJson = gson.toJson(responseObj);

        // Enviar respuesta en formato JSON
        byte[] bytes = responseJson.getBytes(StandardCharsets.UTF_8);
        exchange.getResponseHeaders().set(
            "Content-Type", "application/json"
        );
        exchange.sendResponseHeaders(200, bytes.length);

        OutputStream os = exchange.getResponseBody();
        os.write(bytes);
        os.close();
    }
}

```



```

    }
}

// Estructuras de datos para mapeo de JSON (DTOs)
static class UserData {
    String nombre;
    String correo;
}

static class JsonResponse {
    String status;
    String message;

    JsonResponse(String status, String message) {
        this.status = status;
        this.message = message;
    }
}
}

```

Paso 4: Creación del Frontend (HTML5 + CSS + Fetch API)

Cree la interfaz de usuario en el directorio `frontend/`. Esta interfaz consta de un formulario de registro y una lógica en JavaScript para serializar los campos en un objeto JSON y enviarlo asíncronamente al servidor.

1. Cree el archivo `frontend/index.html` asegurando un anidamiento riguroso de etiquetas de 4 espacios:

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Suscripción al Curso</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="card">
        <h2>Registro de Estudiantes</h2>
        <p>Ingresa sus datos para suscribirse al laboratorio.</p>
    </div>

```

```

    <form id="subscriptionForm">
      <div class="form-group">
        <label for="name">Nombre Completo:</label>
        <input type="text" id="name" required
          placeholder="Ej: Ana Brenes">
      </div>

      <div class="form-group">
        <label for="email">Correo Electrónico:</label>
        <input type="email" id="email" required
          placeholder="ejemplo@correo.com">
      </div>

      <button type="submit">Enviar Registro</button>
    </form>

    <div id="responseContainer"
      class="response-container hidden">
      <p id="responseText"></p>
    </div>
  </div>

  <script>
    const form = document.getElementById(
      'subscriptionForm'
    );

    form.addEventListener('submit', async (e) => {
      e.preventDefault();

      const name = document.getElementById('name').value;
      const email = document.getElementById('email').value;

      // Construir el objeto JavaScript que será el JSON payload
      const payload = {
        nombre: name,
        correo: email
      };

      try {
        // Realizar solicitud HTTP POST asíncrona al backend
        const response = await fetch('/api/submit', {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json'
          }
        });
      } catch (error) {
        console.error('Error al enviar el registro:', error);
      }
    });
  </script>

```

```

        },
        body: JSON.stringify(payload)
    });

    if (response.ok) {
        const data = await response.json();
        showResponse(data.message, false);
    } else {
        showResponse("Error: Código " + response.status, true);
    }
} catch (error) {
    showResponse("Error de red: No se conecta al servidor.", true);
}
});

function showResponse(message, isError) {
    const container = document.getElementById(
        'responseContainer'
    );
    const text = document.getElementById('responseText');

    text.textContent = message;
    container.classList.remove('hidden');

    if (isError) {
        container.classList.add('error');
    } else {
        container.classList.remove('error');
    }
}
</script>
</body>
</html>

```

1. Cree el archivo frontend/style.css aplicando un diseño responsivo minimalista y elegante mediante el uso de variables CSS y una paleta basada en HSL:

```

/* Paleta y Variables de Estilos */
:root {
    --primary: hsl(220, 90%, 56%);
    --bg-dark: hsl(222, 47%, 11%);
    --bg-card: hsl(223, 47%, 16%);
    --text-main: hsl(210, 40%, 98%);
}

```

```

    --text-muted: hsl(215, 20%, 65%);
    --border: hsl(217, 32%, 22%);
    --success: hsl(142, 70%, 45%);
    --error: hsl(0, 84%, 60%);
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  background-color: var(--bg-dark);
  color: var(--text-main);
  font-family: system-ui, -apple-system, sans-serif;
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

.card {
  background-color: var(--bg-card);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 30px;
  width: 90%;
  max-width: 450px;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.3);
}

h2 {
  color: var(--primary);
  font-size: 1.6rem;
  margin-bottom: 8px;
}

p {
  color: var(--text-muted);
  font-size: 0.95rem;
  margin-bottom: 24px;
}

.form-group {
  margin-bottom: 20px;
}

```

```

}

label {
  display: block;
  font-size: 0.9rem;
  margin-bottom: 8px;
  color: var(--text-main);
}

input {
  width: 100%;
  padding: 12px;
  background-color: var(--bg-dark);
  border: 1px solid var(--border);
  border-radius: 6px;
  color: var(--text-main);
  font-size: 1rem;
  outline: none;
  transition: border-color 0.2s, box-shadow 0.2s;
}

input:focus {
  border-color: var(--primary);
  box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.2);
}

button {
  width: 100%;
  padding: 12px;
  background-color: var(--primary);
  border: none;
  border-radius: 6px;
  color: white;
  font-size: 1rem;
  font-weight: bold;
  cursor: pointer;
  transition: background-color 0.2s;
}

button:hover {
  background-color: hsl(220, 90%, 48%);
}

.response-container {
  margin-top: 20px;
  padding: 12px;
}

```

```
border-radius: 6px;
background-color: rgba(59, 130, 246, 0.1);
border: 1px solid var(--primary);
}

.response-container.hidden {
display: none;
}

.response-container.error {
background-color: rgba(239, 68, 68, 0.1);
border-color: var(--error);
color: var(--error);
}
```

Paso 5: Compilación, Ejecución y Enlace de Repositorio Remoto

1. Ingrese a la carpeta del backend y compile el proyecto utilizando Maven. En la raíz de su proyecto, ejecute en su terminal:

```
# Ingresar al directorio del backend
cd backend

# Compilar clases
mvn clean compile
```

1. Ejecute el servidor Java utilizando el plugin de Maven Exec desde la carpeta `backend/` (reemplazando `carnet` por su carné UCR):

```
mvn exec:java \
-Dexec.mainClass="cr.ac.ucr.paraíso.ie.carnet.practica1.AppServer"
```

Abra su navegador e ingrese a <http://localhost:8080>. Digite un nombre y correo electrónico en el formulario y verifique que la respuesta de éxito se imprima en pantalla sin recargar la página.

Vinculación del repositorio remoto en GitHub:

3. Vaya a su cuenta en [GitHub](#) y cree un repositorio público vacío llamado practica-1. **No** lo inicialice con archivos README, gitignore o licencia.
4. Copie la dirección HTTPS provista.
5. Abra una nueva terminal (o regrese al directorio raíz practica-1/ ejecutando `cd ..`) y ejecute los siguientes comandos para vincularlo y subir el código:

```
# Regresar a la raíz del proyecto (si está en la carpeta backend)
cd ..

# Agregar archivos de ambas carpetas (backend y frontend) al staging y confirmar commit
git add .
git commit -m "feat: inicializacion de proyecto con git, maven y servidor java"

# Renombrar rama principal y vincular con origin
git branch -M main
git remote add origin https://github.com/SU_USUARIO/practica-1.git

# Enviar los cambios
git push -u origin main
```

Trampas Comunes

- **Error de Tipo MIME (Estilos no cargados):** Si al abrir el navegador el formulario HTML no tiene estilos CSS aplicados y ve un error en la consola del navegador sobre recursos bloqueados por su tipo MIME, verifique que en su clase `StaticFileHandler` se esté enviando el encabezado `Content-Type` correcto para `.css`. Sin este encabezado específico (`text/css`), muchos navegadores modernos rechazan aplicar los estilos en modo de renderizado estricto.
- **Excepción `BindException: Address already in use`:** Este error ocurre cuando el puerto `8080` ya está siendo utilizado por otro servidor en su computadora (por ejemplo, otra instancia de Tomcat, Node.js o una base de datos). Para solucionarlo, cambie el puerto del servidor en `AppServer.java` (ej. al puerto `8081` o `9090`) y vuelva a compilar con `mvn compile`.
- **Gson lanzando `JsonSyntaxException` en el Backend:** Si el backend lanza una excepción al recibir la solicitud POST, asegúrese de que el cuerpo enviado por el frontend coincida

exactamente en nombre y tipo con los atributos definidos en la clase interna `UserData`. Si el frontend envía `{ "name": "Ana" }` y la clase Java espera `String nombre`, el analizador de Gson no podrá mapear la propiedad de manera automática y asignará valores nulos.

- **Conflicto en Git Push Rejected (Non-fast-forward):** Esto ocurre si el repositorio en GitHub se inicializó con un archivo `README.md` independiente desde la web de GitHub y posee commits que no existen en su repositorio local. Para evitarlo, cree siempre el repositorio de GitHub completamente vacío antes de hacer el primer push.

Pregunta de Reflexión

El servidor Java que usted acaba de codificar maneja las solicitudes HTTP analizando manualmente las cadenas de texto de las rutas y configurando manualmente los encabezados de respuesta y flujos de lectura (`StaticFileHandler` y `ApiHandler`).

Si usted tuviera que agregar 10 endpoints de API REST adicionales y servir 5 carpetas de recursos estáticos diferentes (como imágenes, fuentes e iconos), ¿qué dificultades arquitectónicas y de mantenibilidad enfrentaría con esta estructura nativa y cómo esperaría que un framework empresarial como **Spring Boot** simplifique este proceso?